



Construction d' analyseurs syntaxiques automatiques

c o m p i l a t i o n

 Enseignante: MA. Aida Lahouij

Introduction

- L'analyseur syntaxique (**parser**) reçoit une suite **d'unités lexicales** (de symboles terminaux) de la part de **l'analyseur lexical**.
- Il doit décider si **un flux de tokens** (un mot) est **syntactiquement correct**.
- Autrement dit, il doit décider si c'est un mot qui peut être généré par la grammaire du langage qu'il possède.
- Pour ce faire, il doit construire l'arbre de dérivation de ce mot.
 - S'il y arrive alors le mot est syntaxiquement correct.
 - Sinon, le mot est incorrect.

Introduction

Rappel:

Arbre de dérivation - Arbre Syntaxique

□ Exemple

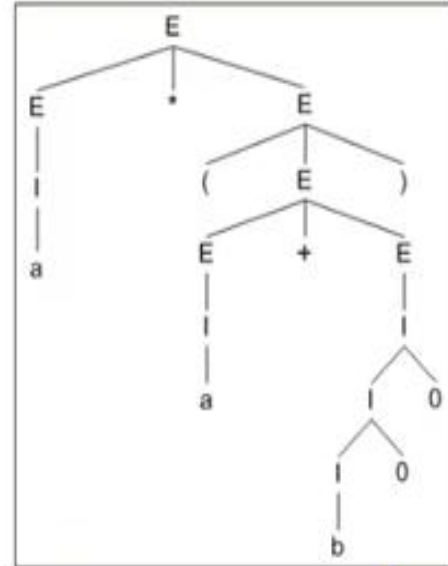
Soit la grammaire suivante : $T = \{a, b, 0, 1, +, *, (,)\}$, $N = \{E, I\}$ avec E est l'axiome, et les règles de production suivantes :

$$E \rightarrow I / E * E / E + E / (E)$$

$$I \rightarrow a / b / I a / I b / I 0 / I 1$$

La figure ci-coté représente l'arbre de dérivation de l'expression : $a * (a + b00)$.
La dérivation la plus à gauche et la plus à droite fournissent le même arbre.

$$\begin{aligned} E &\rightarrow E * E \\ &\rightarrow I * E \\ &\rightarrow a * (E) \\ &\rightarrow a * (E + E) \\ &\rightarrow a * (I + E) \\ &\rightarrow a * (a + E) \rightarrow a * (a + I) \rightarrow a * (a + I 0) \rightarrow a * (a + I 0 0) \rightarrow a * (a + b 0 0) \end{aligned}$$



Introduction

Rappel:

Arbre de dérivation - Arbre Syntaxique

□ Exemple

Soit la grammaire suivante : $T=\{a, b, 0, 1, +, *, (,)\}$, $N=\{E, I\}$ avec E est l'axiome, et les règles de production suivantes :

$$E \rightarrow I / E * E / E + E / (E)$$

$$I \rightarrow a / b / I a / I b / I 0 / I 1$$

▪ Dérivation la plus à gauche de : $a*(b+a0)$

$$E \Rightarrow E * E \Rightarrow I * E \Rightarrow a * E \Rightarrow a * (E) \Rightarrow a * (E + E) \Rightarrow a * (I + E) \Rightarrow a * (b + E) \Rightarrow a * (b + I) \Rightarrow a * (b + I 0) \Rightarrow a * (b + a 0)$$

▪ Dérivation la plus à droite de : $a*(b+a0)$

$$E \Rightarrow E * E \Rightarrow E * (E) \Rightarrow E * (E + E) \Rightarrow E * (E + I) \Rightarrow E * (E + I 0) \Rightarrow E * (E + a 0) \Rightarrow E * (I + a 0) \Rightarrow E * (b + a 0) \Rightarrow I * (b + a 0) \Rightarrow a * (b + a 0)$$

Analyse syntaxique avec bison:

Bison est un outil de génération automatique d'analyseurs syntaxiques.

Un analyseur syntaxique écrit en Bison consiste en une description d'une GHC (Grammaire Hors Conflit). Un programme Bison est un simple fichier texte enregistré avec l'extension ".y". Il y a plusieurs familles de GHC :

Famille	Abréviation complète	Sens
LL(1)	Left-to-right, Leftmost derivation (1 lookahead)	Lecture de gauche à droite, dérivation la plus à gauche, 1 symbole d'anticipation
LL(k)	Left-to-right, Leftmost derivation (k lookahead)	Idem, mais avec k symboles d'anticipation
SLR	Simple LR	Version simplifiée de LR(1) avec des tables plus petites
LALR(1)	Look-Ahead LR (1 lookahead)	Version optimisée de LR(1), plus compacte, utilisée par Yacc/Bison
LALR(k)	Look-Ahead LR (k lookahead)	Comme LALR(1) mais avec plus de symboles d'anticipation
GLR	Generalized LR	Gère les grammaires ambiguës, analyse plusieurs chemins en parallèle

-Bison fait partie de la famille **LALR(1)**, c'est-à-dire **Look-Ahead LR** avec 1 symbole d'anticipation.

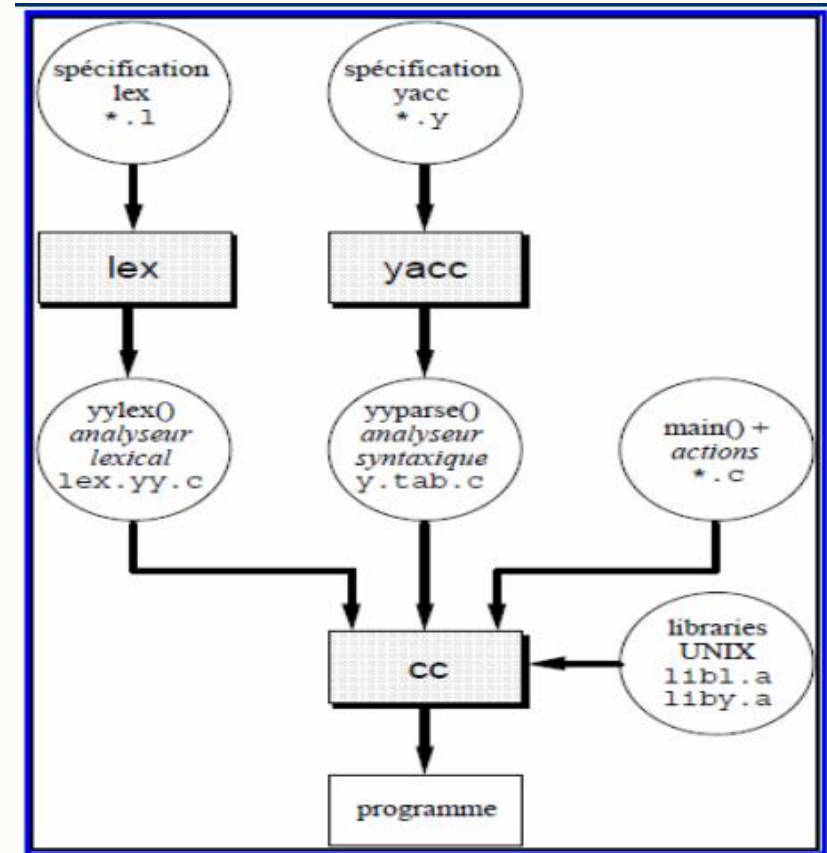
Analyse syntaxique avec bison:

BISON est un compilateur qui produit automatiquement un analyseur syntaxique à partir d'un schéma de traduction dirigée par la syntaxe (**grammaire + actions sémantiques**).

L'analyseur syntaxique ainsi produit, reconnaîtra les mots du langage engendré par la grammaire donnée.

L'analyseur syntaxique est réalisé par une fonction « C » générée par Bison.

Cette fonction porte le nom de « **yyparse()** ».



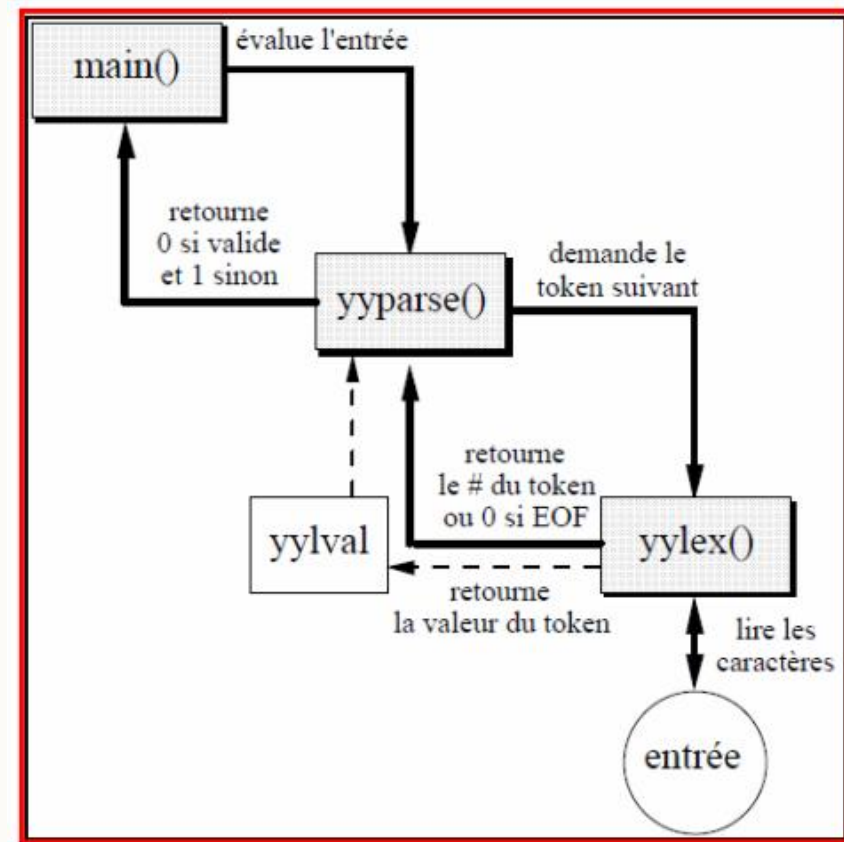
Analyse syntaxique avec bison:

La fonction « **yyparse()** » appelle à chaque fois la fonction « **yylex()** » pour obtenir le prochain lexème de l'entrée.

La fonction « **yylex()** » peut être soit codée manuellement en « C », soit générée automatiquement par le compilateur Flex.

La fonction « **yyparse()** » retourne un entier :

- **0**, si l'analyse syntaxique a été effectuée avec succès.
- **1**, si l'analyseur syntaxique a rencontré au moins une erreur de syntaxe



Analyse syntaxique avec bison:

main() :

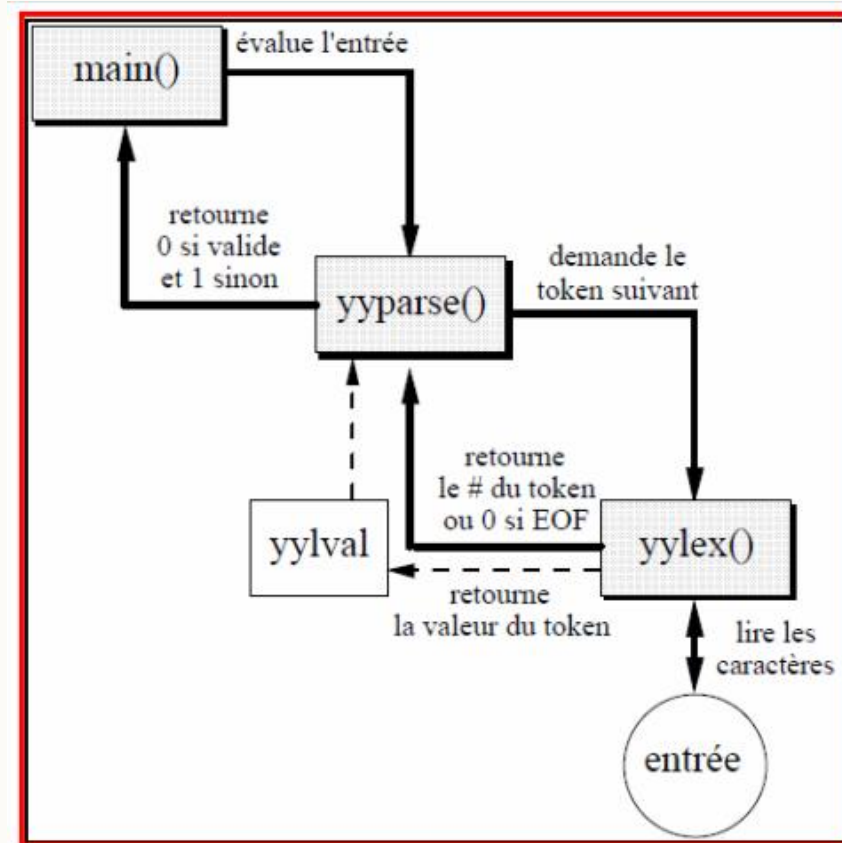
- Fonction principale qui évalue l'entrée et lance l'analyse syntaxique avec yyparse().
- Retourne 0 si l'analyse est valide, 1 sinon.

yyparse() (Bison - analyse syntaxique) :

- C'est l'analyseur syntaxique généré par Bison.
- Il demande le token suivant à yylex() pour construire l'arbre syntaxique.

yylex() (Flex - analyse lexicale) :

- Fonction générée par Flex qui lit l'entrée caractère par caractère.
- Elle identifie des tokens et les envoie à yyparse().
- Retourne un **numéro de token** ou **0 si EOF** (fin de fichier).



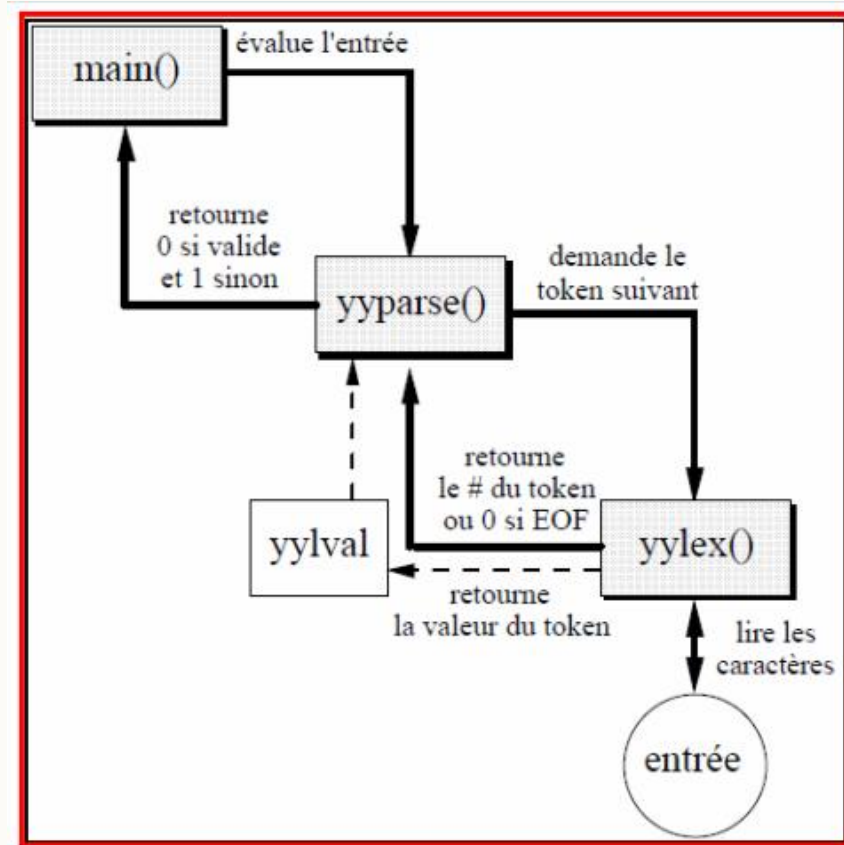
Analyse syntaxique avec bison:

yylval :

- Une variable qui stocke la valeur du token reconnu par yylex().
- Elle est utilisée par yyparse() pour l'analyse syntaxique.

Entrée :

- Les données brutes (code source, texte, etc.) qui sont analysées.
- yylex() lit ces données et en extrait des tokens.



Analyse syntaxique avec bison:

Étapes de compilation à suivre pour réaliser un analyseur syntaxique à partir de Bison :

- Compiler le fichier de spécification « **.y** » de l'analyseur **syntaxique** avec la commande « **bison** ».
- Compiler le fichier de spécification « **.l** » de l'analyseur **lexical** avec la commande « **flex** ».
- Compiler les deux analyseurs avec un compilateur C (gcc, par exemple).

Analyse syntaxique avec bison:

Exemple :

- Soient les fichiers de spécification :
« **parser.y** » de l'analyseur syntaxique.
« **scan.l** » de l'analyseur lexicale.

Etapas de compilation :

1. Génération du fichier Bison

```
bison -d parser.y
```

Cette commande génère deux fichiers :

parser.tab.c → Contient le code C de l'analyseur syntaxique.

parser.tab.h → Contient les définitions des tokens pour l'analyse lexicale.

Analyse syntaxique avec bison:

2.Génération du fichier flex

flex scan.l

Génère un fichier lex.yy.c contenant l'analyseur lexical.

yylex() est défini dans ce fichier et utilisé par parser.tab.c.

Analyse syntaxique avec bison:

3 : Compilation finale

```
gcc -o compiler parser.tab.c lex.yy.c -ly -lfl
```

Génère un exécutable appelé compiler.

Options de compilation :

-ly → Lie la bibliothèque Bison (Yacc).

-lfl → Lie la bibliothèque Flex.

parser.tab.c → Contient l'analyse syntaxique.

lex.yy.c → Contient l'analyse lexicale.

Analyse syntaxique avec bison:

3 : Exécution du compilateur

compiler <source >cible

source : Fichier d'entrée (code à analyser ou compiler).

cible : Fichier de sortie (résultat après analyse ou compilation).

Analyse syntaxique avec bison:

Format du fichier de spécification

Un fichier de spécification de Bison possède 3 sections :

`%{`

Déclarations

`%}`

Définitions

`%%`

Règles

`%%`

Code complémentaire

Analyse syntaxique avec bison:

Format du fichier de spécification

Les déclarations :

- Un bloc délimité par les symboles « %{ » et « %} » contient les déclarations C des variables et des fonctions globales, ainsi que les directives du pré-processeur.
- Ces déclarations peuvent être utilisées dans les sections 2 et 3.

```
%{  
    #include <stdio.h>  
    #include <stdlib.h>  
%}
```


Analyse syntaxique avec bison:

Format du fichier de spécification

Les définitions:

- Les tokens doivent être également déclarés dans cette section.
- Les tokens peuvent être soit nommés, soit données sous forme de chaînes terminales entre deux quotes '...'.
'...'

Les tokens nommés

```
%token PLUS MINUS NUMBER
%%
expr: expr PLUS expr { $$ = $1 + $3; }
    | expr MINUS expr { $$ = $1 - $3; }
    | NUMBER { $$ = $1; }
    ;

"+" { return PLUS; }
"- " { return MINUS; }
[0-9]+ { yylval = atoi(yytext); return NUMBER; }
```

Les tokens entre '....'

```
%%
expr: expr '+' expr { $$ = $1 + $3; }
    | expr '-' expr { $$ = $1 - $3; }
    | 'NUMBER' { $$ = $1; }
    ;
```

```
[0-9]+ { yylval = atoi(yytext); return NUMBER; } //
[+\-] { return yytext[0]; } // Reconnaissance des
```

Les tokens nommés sont plus clairs pour de grands projets, mais les chaînes terminales sont plus simples pour des petites grammaires.

Analyse syntaxique avec bison:

Format du fichier de spécification

Les définitions:

La forme d'une déclaration de tokens est :

%token **token1** **value1** **token2** **value2** . . .

```
%token NUMBER 256
```

```
%token PLUS 257 MINUS 258 MULT 259 DIV 260
```

```
%token LPAREN 261 RPAREN 262
```

```
%token END 263
```

- Les valeurs entières définissent les codes des tokens utilisés par l'analyseur lexical.
- Tous les tokens déclarés doivent avoir des valeurs positives comme code.

L'affectation des valeurs de code pour les tokens est optionnel : les tokens n'ayant pas un code explicite reçoivent des codes implicites.

Analyse syntaxique avec bison:

Format du fichier de spécification

Les définitions:

La déclaration de l'axiome de la grammaire est définie par la commande :

%start nom_axiome

- Par défaut, le nom de la tête de la première production est l'axiome.
- La « **%start** » est donc obligatoire lorsque la première production écrite n'est pas celle issue de l'axiome de la grammaire.

Analyse syntaxique avec bison:

Format du fichier de spécification

Les règles:

- Une règle correspond à une production étendue de la grammaire.
- La forme d'une règle dans Bison est :

Tête : **Queue** **Actions**;

- **Tête** : est le nom du symbole variable se trouvant à gauche d'une production de la grammaire.
- **Queue** : est la forme se trouvant à droite dans une production.
- **Actions** : sont des fragments de programmes C qui seront exécutées par la fonction « **yyparse()** » lorsque la règle aura été reconnue.
- Dans **Actions**, on peut faire référence aux variables globales déclarées dans la 1ère section, appeler des sous-programmes déclarés dans la 3ème section, etc.

Analyse syntaxique avec bison:

Format du fichier de spécification

Les règles:

Écriture des productions en Bison :

Soient « $X \rightarrow A \mid B \mid C \mid \epsilon$ » les règles de production dont la tête est le symbole « X ».

– En Bison, ces règles s'écrivent :

$X : A ;$

$X : B ;$

$X : C ;$

$X : ;$

– L'ordre des productions n'est pas obligatoire

– On peut également les écrire d'une manière simplifiée en utilisant le symbole d'union « $\mid$ » :

$X : A ;$

$\mid B ;$

$\mid C ;$

$\mid ;$

Analyse syntaxique avec bison:

Format du fichier de spécification

Les règles:

- La notation **\$** permet de manipuler les valeurs des symboles dans les règles de Bison.
- **\$\$** représente la valeur du côté gauche de la production.
- **\$1, \$2, \$3**, etc., correspondent aux éléments du côté droit dans l'ordre.
- Cette notation est essentielle pour évaluer des expressions et construire un interpréteur.

Exemple :

Soit la production « $E \rightarrow T "+" F$ ».

- La valeur de l'expression « E » est celle de « T » plus celle de « F ».
- On peut alors écrire en Bison l'action sémantique suivante :

$E : T "+" F \{ \$\$ = \$1 + \$3; \};$

Analyse syntaxique avec bison:

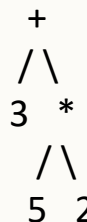
Format du fichier de spécification

Les règles:

- La notation $\$$ est utilisée dans les **actions sémantiques de Bison** pour attribuer des valeurs aux symboles non-terminaux et réaliser des calculs ou des transformations pendant l'analyse syntaxique.

Sans la notation $\$$, il serait impossible d'évaluer une expression comme $3 + 5 * 2$.

Exemple : analyser $3 + 5 * 2$



1. Faire le calcul: On doit comprendre que $5 * 2$ se fait avant, donc le résultat est : $3 + (5 * 2) = 3 + 10 = 13$
2. Garder des valeurs en mémoire: On garde 3, 5 et 2 pendant qu'on lit l'expression, pour les utiliser au bon moment.
3. Faire des actions selon les règles: Si on lit une addition, on additionne ; si on lit une multiplication, on multiplie.
4. Construire une sorte d'arbre: On peut représenter l'expression avec un arbre comme ça :

Analyse syntaxique avec bison:

Format du fichier de spécification

Le code complémentaire :

- La 3ème section comprend une séquence de déclarations de fonctions « C » utilisées par ailleurs dans les actions ou par l'analyseur généré.
- L'une d'entre elles doit être la fonction « **yyparse()** ».
- Cette fonction délivre à chaque appel un entier qui est le code de l'unité lexicale suivante lue, et transmet dans la variable **yyval** la valeur associée à cette unité lexicale.
- Par défaut, le type de la variable **yyval** est « **int** ».
- Il est possible de modifier le type de la variable **yyval** via la macro-constante :

```
#define YYSTYPE char*
```

Toutes les valeurs de mes symboles seront des chaînes de caractères (char*)

Analyse syntaxique avec bison:

- L'analyseur syntaxique travaille avec une pile : il empile et dépile les symboles (terminaux et variables) avec leurs attributs.
- Une clause « **%union** » permet de changer les types de ces deux variables.
- Lorsqu'un lexème n'a pas d'attribut ou a un attribut entier, il n'est pas nécessaire de spécifier le type (car le type entier est pris par défaut).
- La clause **%type** permet de déclarer les noms et les types **des symboles non terminaux**.
- De même, lorsqu'un symbole variable n'a pas d'attribut ou a un attribut entier, il n'est pas nécessaire de spécifier le type.
- Les types spécifiés par **%token** et **%type** doivent avoir été définis par une clause **%union**

Analyse syntaxique avec bison:

Définir les attributs et leurs types :

- La structure contenant les attributs d'un symbole (terminal ou variable) est décrite à l'aide de la clause « **%union** ».
- Deux variables globales sont fournies (de type « int » par défaut) :
yylval : l'analyseur lexical stocke dans cette variable les attributs de l'unité lexicale reconnue.
- L'analyseur syntaxique récupère ces attributs en lisant cette variable.

Analyse syntaxique avec bison:

Déclaration des Types d'Attributs avec %union

Dans Bison, on utilise la directive **%union** pour définir les types de données utilisés par les symboles.

Exemple:

```
%union {  
    int ival;    /* Pour stocker un entier */  
    float fval;  /* Pour stocker un nombre flottant */  
    char* sval; /* Pour stocker une chaîne de caractères */  
}
```

Cela permet de dire que chaque symbole peut avoir différents types d'attributs.

Analyse syntaxique avec bison:

Déclaration des Types pour les Symboles (%token et %type)

Une fois **%union** défini, on doit associer un type aux tokens (terminaux) et aux variables (non-terminaux).

Exemple:

```
%token <ival> INTEGER      /* Token INTEGER aura un attribut de type int */  
%token <fval> FLOAT        /* Token FLOAT aura un attribut de type float */  
%type <ival> exp term      /* Les non-terminaux exp et term auront des entiers */
```

Cela permet de dire que chaque symbole peut avoir différents types d'attributs.

Analyse syntaxique avec bison:

Informations sur les opérateurs :

- **%left** : permet de spécifier des opérateurs associatifs à gauche.

%left : Associativité à gauche

Utilité : Indique que l'opérateur s'évalue de gauche à droite.

Exemple : Les opérateurs **+** et **-** sont associatifs à gauche, donc :

- $10 - 4 - 2$ est évalué comme $(10 - 4) - 2 = 4$.

```
%left '+' '-' /* Déclare + et - comme associatifs à gauche */
```

Sans %left, $1 - 2 - 3$ pourrait être ambigu (interprété comme $1 - (2 - 3) = 2$ au lieu de $(1 - 2) - 3 = -4$).

Analyse syntaxique avec bison:

Informations sur les opérateurs :

- **%right** : permet de spécifier des opérateurs associatifs à droite.

%right : Associativité à droite

Utilité : Indique que l'opérateur s'évalue de droite à gauche.

Exemple : L'opérateur `=` est associatif à droite :

- `a = b = c` est interprété comme `a = (b = c)`.

```
%right '=' /* L'opérateur = est associatif à droite */
```

Sans `%right`, `a = b = c` pourrait être mal interprété comme `(a = b) = c`, ce qui est incorrect en C.

Analyse syntaxique avec bison:

Informations sur les opérateurs :

- **%noassoc** : permet de spécifier des opérateurs non associatifs

%noassoc : Opérateurs non associatifs

Utilité : Spécifie que l'opérateur ne peut pas être utilisé deux fois de suite.

Exemple : Les opérateurs `<`, `>` (comparaisons) sont non associatifs car `3 < 4 < 5` n'a pas de sens.

```
%noassoc '<' '>' /* Déclare < et > comme non associatifs */
```

Sans %noassoc, `3 < 4 < 5` pourrait être interprété de manière imprévue.

Analyse syntaxique avec bison:

Informations sur les opérateurs :

- **%prec** : permet de forcer la précedence des opérateurs.

%prec : Forcer la précedence d'un opérateur

Utilité : Change la priorité d'un opérateur dans une règle sans affecter les autres.

Exemple :

- On veut que '-' unaire (ex: -5) ait plus de priorité que - binaire (ex: 10 - 2).
- On force sa priorité avec %prec .

Analyse syntaxique avec bison:

Informations sur les opérateurs :

- **%prec** : permet de forcer la précedence des opérateurs.

```
%left '+' '-'
%left '*' '/'
%right UMINUS /* Précedence plus haute que - binaire */

%%

exp: exp '+' exp { $$ = $1 + $3; }
    | exp '-' exp { $$ = $1 - $3; }
    | exp '*' exp { $$ = $1 * $3; }
    | exp '/' exp { $$ = $1 / $3; }
    | '-' exp %prec UMINUS { $$ = -$2; } /* Précedence plus haute */
    | NUMBER
    ;
```

Analyse syntaxique avec bison:

Comment récupérer les valeurs dans l'analyseur syntaxique

Dans les règles de grammaire (.y), on utilise \$1, \$2, etc. pour accéder aux attributs des symboles.

Exemple :

```
exp: exp '+' term { $$ = $1 + $3; } /* Utilisation des attributs */
    | term          { $$ = $1; }
    ;

term: INTEGER      { $$ = $1; } /* INTEGER a une valeur stockée dans yylval.ival */
    | FLOAT        { $$ = (int) $1; } /* Conversion de FLOAT en int */
    ;
```

\$1 représente le premier élément de la règle.

\$3 représente le troisième élément.

\$\$ est le résultat de la règle, qui est renvoyé à l'analyseur syntaxique.

Analyse syntaxique avec bison:

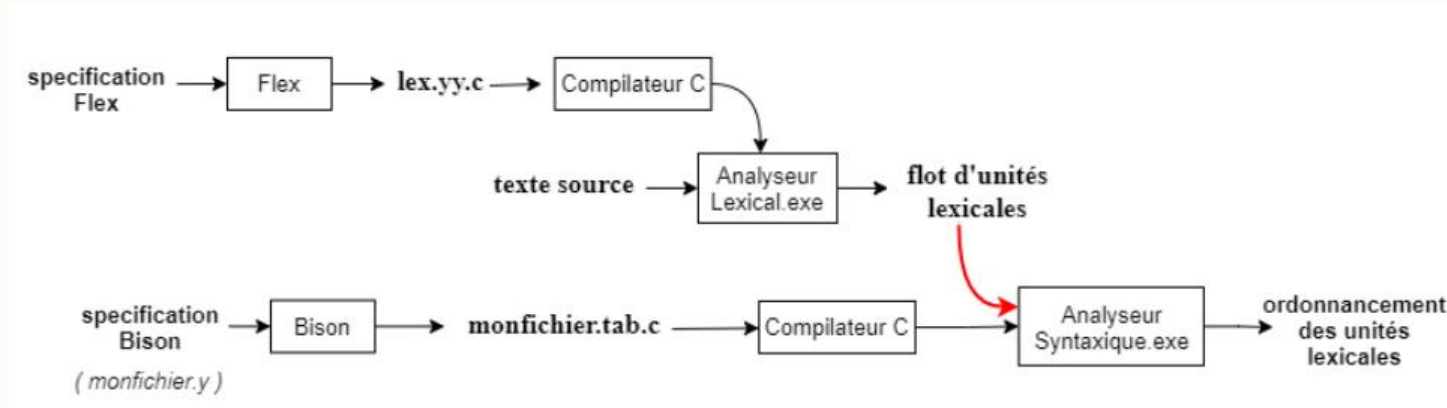
Procédure de récupération sur erreur :

- On peut spécifier une fonction de traitement d'erreurs dans un analyseur syntaxique écrit à l'aide de Bison.
- Cette fonction doit porter le nom « **yyerror()** » pour être invoquée automatiquement lorsqu'une erreur syntaxique est rencontrée.

Exemple :

```
void yyerror(char *s)
{
    fprintf(stderr, "%s\n", s);
    return 0;
}
```

Analyse syntaxique avec bison:



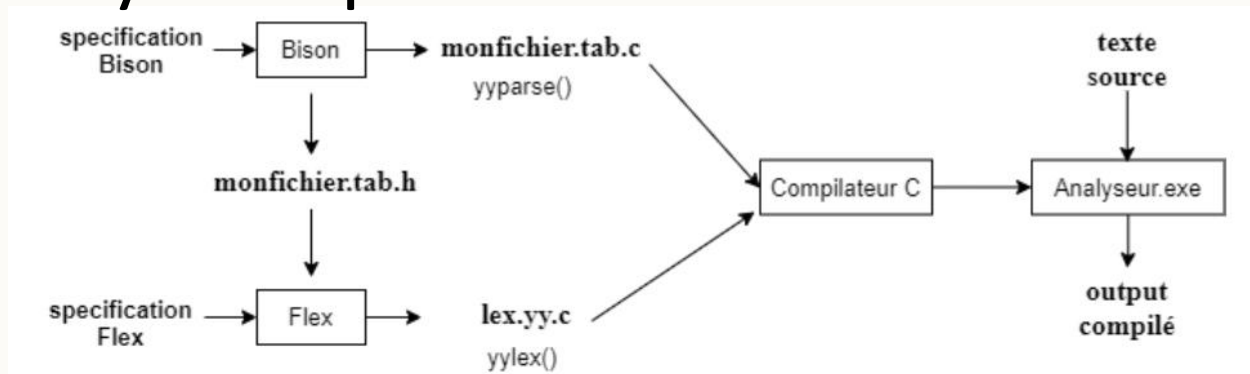
Pour fournir l'output de l'analyseur lexical comme input à l'analyseur syntaxique, il faut procéder à une **coordination entre les deux analyses**.

Cette coordination est illustrée par une flèche rouge sur la figure ci-dessus.

En pratique, **pour combiner les deux analyses**, il faut :

- 1) générer un fichier en-tête de l'analyseur syntaxique qui définit les tokens par une commande *%token*,
- 2) et inclure ce fichier dans le fichier .l

Analyse syntaxique avec bison:



Dans le fichier de spécification *monfichier.l*, il faut donc ajouter les informations suivantes dans la partie déclaration :

```
#include "monfichier.tab.h"  
extern int yyval ;
```

Pour que bison génère un fichier *.h* en plus du code source en C, il faut utiliser l'option *-d*. La compilation s'effectue par conséquent comme suit :

- *bison -d monfichier.y*
- *flex monfichier.l*
- *gcc -o analyseur lex.yy.c y.tab.c*

Analyse syntaxique avec bison:

Exemple:Un calculateur simple

L'analyseur lexical :

```
/* fichier calc.lex */
%{
    /* fichier dans lequel est défini NOMBRE */
    #include "calc.tab.h"
}%
%%
[0-9]+ { yylval = atoi(yytext); return NOMBRE;}
[ \t]  ; /* ignore les blancs et tabulations */
\n     return 0;
.      return yytext[0];
%%
```

Analyse syntaxique avec bison:

Exemple:Un calculateur simple

L'analyseur syntaxique :

```
/* fichier calc.y */
```

```
%{
```

```
    #include<stdio.h>
```

```
%}
```

```
%token NOMBRE
```

```
%start calcul
```

```
%%
```

```
calcul : expr {printf("Resultat = %d\n« , $1);}
```

```
;
```

Analyse syntaxique avec bison:

Exemple:Un calculateur simple

L'analyseur syntaxique :

```
expr : expr '+' terme {$$ = $1 + $3;}  
    | expr '-' terme {$$ = $1 - $3;}  
    | terme {$$ = $1;}  
    ;  
terme: terme '*' facteur {$$ = $1 * $3;}  
    | terme '/' facteur {$$ = $1 / $3;}  
    | facteur {$$ = $1;}  
    ;
```

```
facteur: '(' expr ')' {$$ = $2;}  
    | '-' facteur {$$ = -$1;}  
    | NOMBRE {$$ = $1;}  
    ;  
%%  
int yyerror()  
{  
    fprintf(stderr, "erreur de syntaxe\n");  
    return 1;  
}
```


Analyse syntaxique avec bison:

Exemple: Un calculateur simple

Compilation :

```
bison -d calc.y
```

```
flex calc.lex
```

```
gcc -o calc calc.tab.c lex.yy.c -ly -lfl
```

```
c:\FLexBison>calc
1+(2*3)
Resultat = 7

c:\FLexBison>calc
(5-2)*(5+2)
Resultat = 21

c:\FLexBison>calc
28/(3+4)
Resultat = 4

c:\FLexBison>calc
1+(5-2
erreur de syntaxe
```

THANK YOU!

